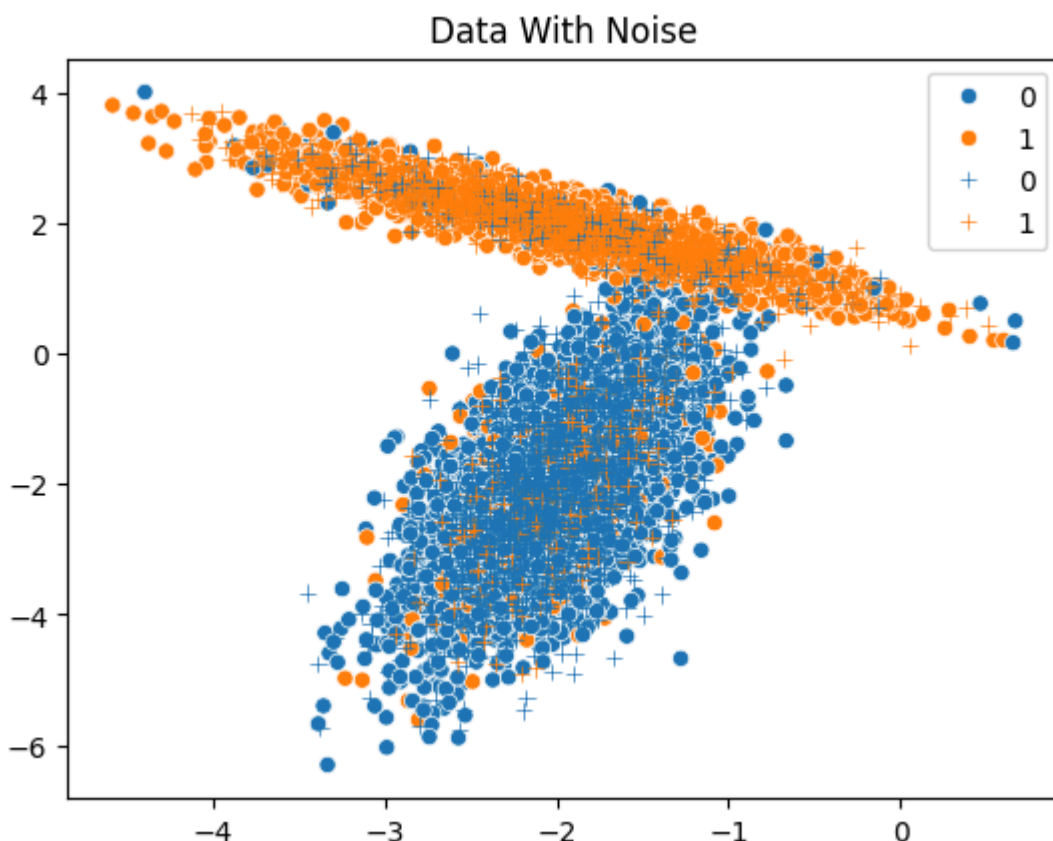


```

from sklearn.datasets import make_classification # для генерации данных
from sklearn.model_selection import train_test_split # для разделения на
from sklearn.ensemble import RandomForestClassifier # случайный лес
import numpy as np
import seaborn as sns # для простого отображения
import matplotlib.pyplot as plt #
# создаем данные
X,y = make_classification(n_samples=10000, # число примеров
    n_features=2, # число признаков (атрибутов)
    n_informative=2, # из них информативных
    n_redundant=0, # из них не информативных
    n_repeated=0, # из них повторяющихся
    n_classes=2, # число классов
    n_clusters_per_class=1, # число кластеров на класс
    class_sep=2, # влияет на расстояние между кластерами
    flip_y=0.2, # 0.2 доля ошибок (шума)
    weights=[0.5,0.5], # пропорции числа данных в классах
    random_state=17) #
# разделяем на обучающие и тестовые, случайно
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.33, random_state=42)
# рисуем данные
plt.subplots();#
ax1=plt.gca();#
sns.scatterplot(x=X_train[:,0],y=X_train[:,1],hue=y_train,ax=ax1);# обучающие
sns.scatterplot(x=X_test[:,0],y=X_test[:,1],hue=y_test,ax=ax1,marker="+")# тестовые
ax1.set_title("Data With Noise");#
plt.show();#

```



```

clf = RandomForestClassifier(max_depth=5 # 5 максимальная глубина деревьев

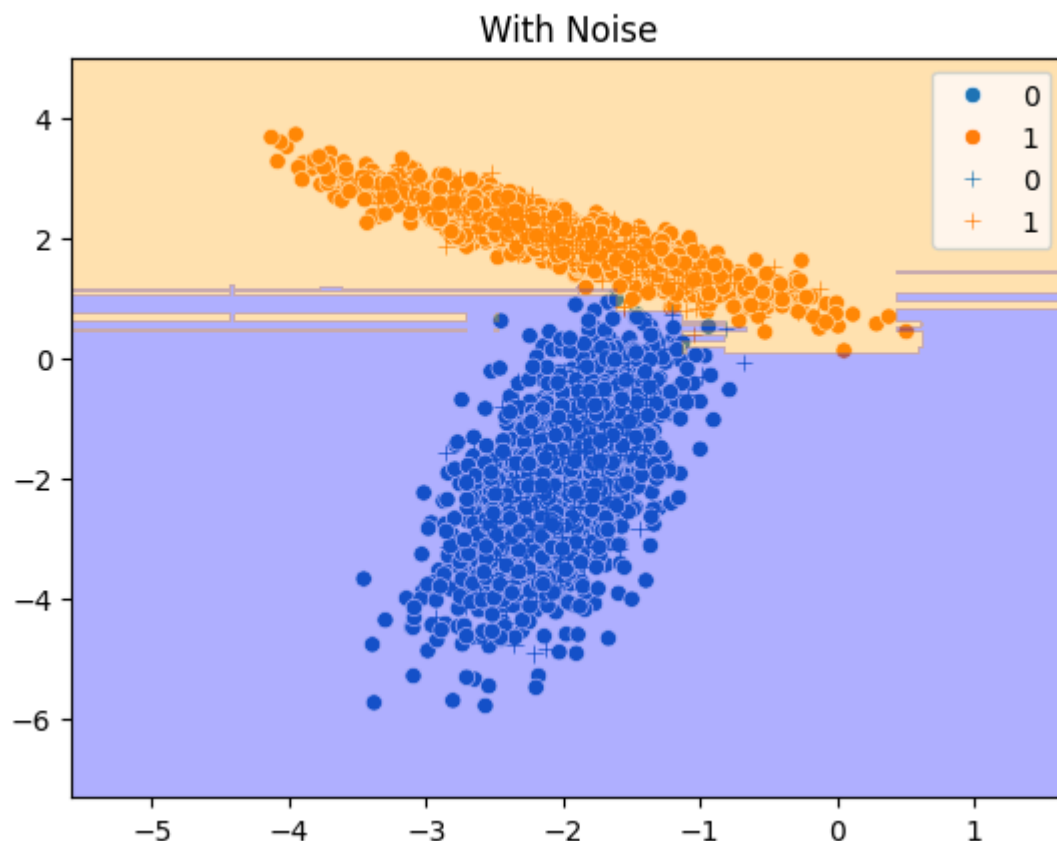
```

```

clf = RandomForestClassifier(max_depth=5, n_estimators=100) # 5 максимальная глубина дерева
n_estimators=5, # 10 число деревьев в лесу
max_features=1) # максимальное число признаков для каждого дерева
#from sklearn.tree import DecisionTreeClassifier
#clf=DecisionTreeClassifier(max_depth=5)
clf.fit(X_train, y_train) # обучаем
y_pred = clf.predict(X_test) # проверяем на тестовых данных
score=clf.score(X_test, y_test) # считаем среднюю точность
print(score)
ind=y_test==y_pred; # индексы совпадений результата классификации и меток
plt.subplots();
ax2=plt.gca();
# рисуем "правильно" распознанные примеры
sns.scatterplot(x=X_test[ind,0],y=X_test[ind,1],hue=y_test[ind],ax=ax2).
# рисуем "неправильно" распознанные примеры
sns.scatterplot(x=X_test[~ind,0],y=X_test[~ind,1],hue=y_pred[~ind],ax=ax2).
#sns.scatterplot(x=X_test[:,0],y=X_test[:,1],hue=y_test[:,],ax=ax2,marker='x')
ax2.set_title("With Noise");
# считаем и рисуем разделяющую поверхность.
plot_step=0.01
x_min, x_max = X[:, 0].min() - 1, X[:, 0].max() + 1 # немного измененные границы
y_min, y_max = X[:, 1].min() - 1, X[:, 1].max() + 1 # немного измененные границы
# считаем прямоугольную сетку возможных значений этих атрибутов
xx, yy = np.meshgrid(np.arange(x_min, x_max, plot_step), #
                     np.arange(y_min, y_max, plot_step)) #
Z = clf.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape) # и преобразуем обратно в исходному размеру
cs = plt.contourf(xx, yy, Z, levels=1, colors=['blue','orange'],alpha=0.5)
plt.show();

```

0.8951515151515151



```

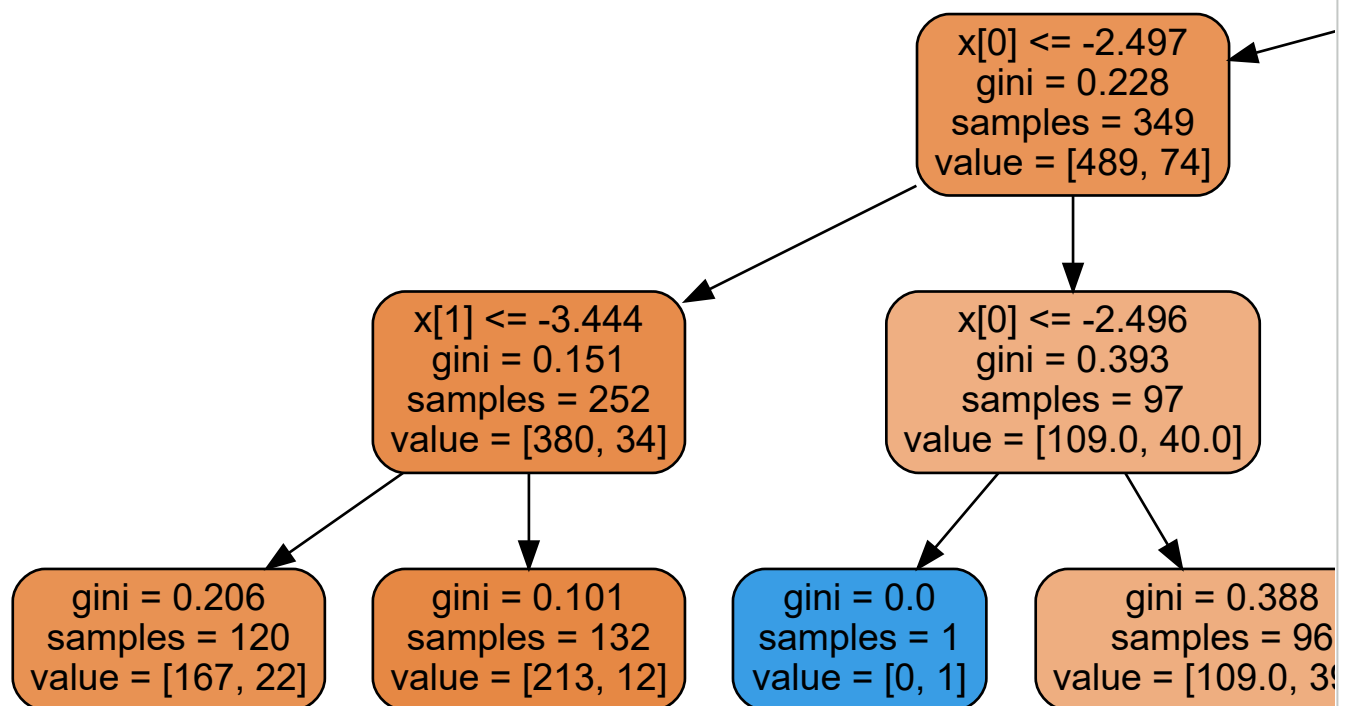
tree_data=clf.estimators_[2] # третье дерево
# рисуем его
from sklearn import tree

```

```

import graphviz
dot_data = tree.export_graphviz(tree_data, out_file=None, # можем указать
    filled=True, rounded=True) # прочие детали отображения
graph = graphviz.Source(dot_data) # загружаем дерево из переменной или из файла
graph

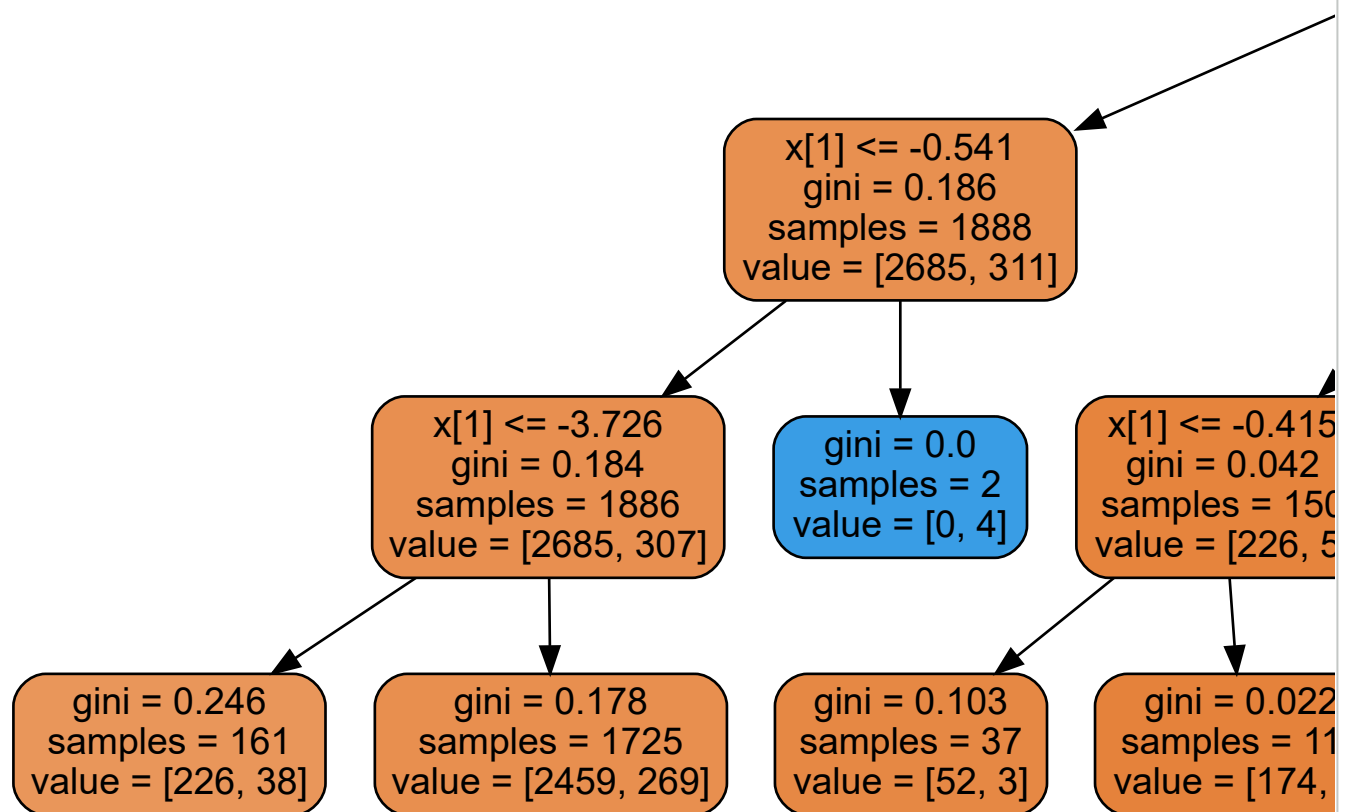
```



```

tree_data=clf.estimators_[4] # третье дерево
# рисуем его
from sklearn import tree
import graphviz
dot_data = tree.export_graphviz(tree_data, out_file=None, # можем указать
    filled=True, rounded=True) # прочие детали отображения
graph = graphviz.Source(dot_data) # загружаем дерево из переменной или из файла
graph # отображаем на экране

```



```

from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier # Added this missing
from sklearn.metrics import accuracy_score, precision_score, recall_score
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

```

```

# Load data
data_wine = load_wine()
print("Features: ", data_wine.feature_names)
print("Labels: ", data_wine.target_names)

```

```

X = data_wine.data
y = data_wine.target

```

```

# Split into train and test

```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3)

# Create and train classifier
clf = RandomForestClassifier(max_depth=5, n_estimators=10, max_features:
clf.fit(X_train, y_train)

# Predict on test data
pred = clf.predict(X_test)

# Calculate metrics
accuracy = accuracy_score(y_test, pred)
f1 = f1_score(y_test, pred, average="macro")
precision = precision_score(y_test, pred, average="macro")
recall = recall_score(y_test, pred, average="macro")

print('Accuracy=', accuracy)
print('F1=', f1)
print('Precision=', precision)
print('Recall=', recall)

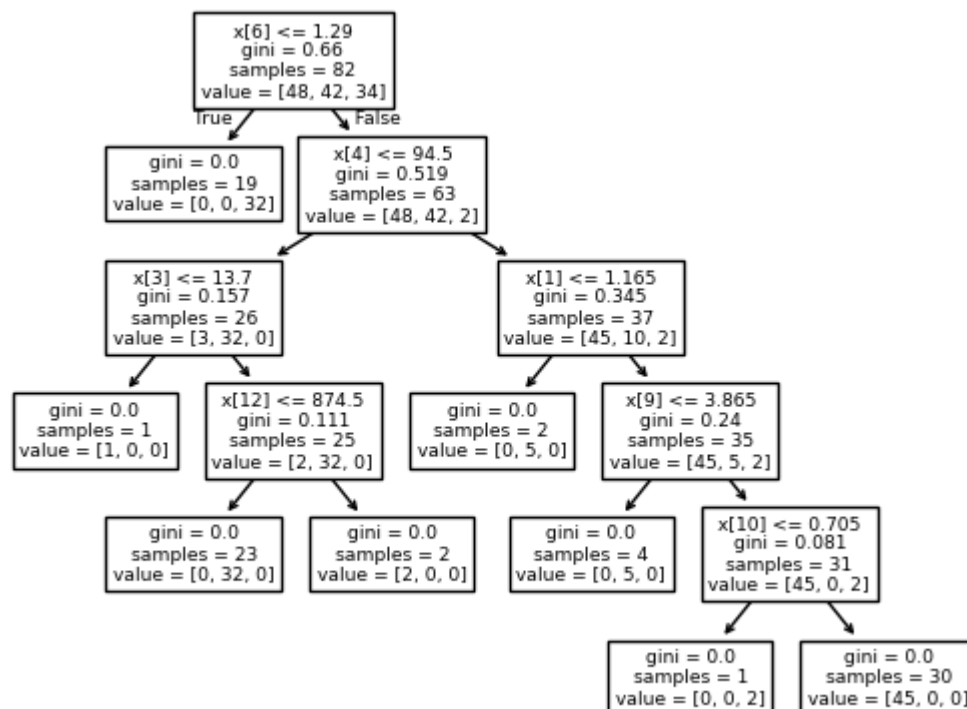
# Define the function (unchanged)
def plot_confusion_matrix(clf, x, y):
    predictions = clf.predict(x)
    cm = confusion_matrix(y, predictions, labels=clf.classes_)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=c
    disp.plot()
    plt.show()

# Call the function correctly (fixed the call)
plot_confusion_matrix(clf, X_test, y_test)
```

Features: ['alcohol', 'malic_acid', 'ash', 'alcalinity_of_ash', 'magnesium', 'total_ph']
 Labels: ['class 0', 'class 1', 'class 2']

```
estimator = clf.estimators_[2]
# рисуем еро
import graphviz
from sklearn import tree
tree.plot_tree(estimator)
```

```
[Text(0.3, 0.9166666666666666, 'x[6] <= 1.29\ngini = 0.66\nsamples = 82\nvalue = [48, 42, 34]'),
Text(0.2, 0.75, 'gini = 0.0\nsamples = 19\nvalue = [0, 0, 32]'),
Text(0.25, 0.8333333333333333, 'True '),
Text(0.4, 0.75, 'x[4] <= 94.5\ngini = 0.519\nsamples = 63\nvalue = [48, 42, 2]'),
Text(0.35, 0.8333333333333333, ' False'),
Text(0.2, 0.5833333333333334, 'x[3] <= 13.7\ngini = 0.157\nsamples = 26\nvalue = [3, 32, 0]'),
Text(0.1, 0.4166666666666667, 'gini = 0.0\nsamples = 1\nvalue = [1, 0, 0]'),
Text(0.3, 0.4166666666666667, 'x[12] <= 874.5\ngini = 0.111\nsamples = 25\nvalue = [2, 32, 0]'),
Text(0.2, 0.25, 'gini = 0.0\nsamples = 23\nvalue = [0, 32, 0]'),
Text(0.4, 0.25, 'gini = 0.0\nsamples = 2\nvalue = [2, 0, 0]'),
Text(0.6, 0.5833333333333334, 'x[1] <= 1.165\ngini = 0.345\nsamples = 37\nvalue = [45, 10, 2]'),
Text(0.5, 0.4166666666666667, 'gini = 0.0\nsamples = 2\nvalue = [0, 5, 0]'),
Text(0.7, 0.4166666666666667, 'x[9] <= 3.865\ngini = 0.24\nsamples = 35\nvalue = [45, 5, 2]'),
Text(0.6, 0.25, 'gini = 0.0\nsamples = 4\nvalue = [0, 5, 0]'),
Text(0.8, 0.25, 'x[10] <= 0.705\ngini = 0.081\nsamples = 31\nvalue = [45, 0, 2]'),
Text(0.7, 0.08333333333333333, 'gini = 0.0\nsamples = 1\nvalue = [0, 0, 2]'),
Text(0.9, 0.08333333333333333, 'gini = 0.0\nsamples = 30\nvalue = [45, 0, 0]')]
```



```
tree_data=tree.export_graphviz(estimator, out_file=None,
feature_names = data_wine.feature_names,
class_names = data_wine.target_names,
```

```
rounded = True, proportion = False,  
precision = 2, filled = True);  
graph=graphviz.Source(tree_data,format='png');  
graph
```

